

Learning discriminant functions with fuzzy attributes for classification using genetic programming

Been-Chian Chien^a(錢炳全), Jung-Yi Lin^a(林忠億),
Tzung-Pei Hong^b(洪宗貝)

^aInstitute of Information Engineering, I-Shou University, 1, Section 1, Hsueh-Cheng Road, Ta-Hsu Hsiang Kaohsiung County, 84008 Taiwan, ROC

^bDepartment of Electrical Engineering, National University of Kaohsiung, 251, Lane 280, Der-Chung Road, Nan-Tzu District, Kaohsiung City, 811 Taiwan, ROC

Expert Systems with Applications, Vol. 23(2002), pp. 31-37

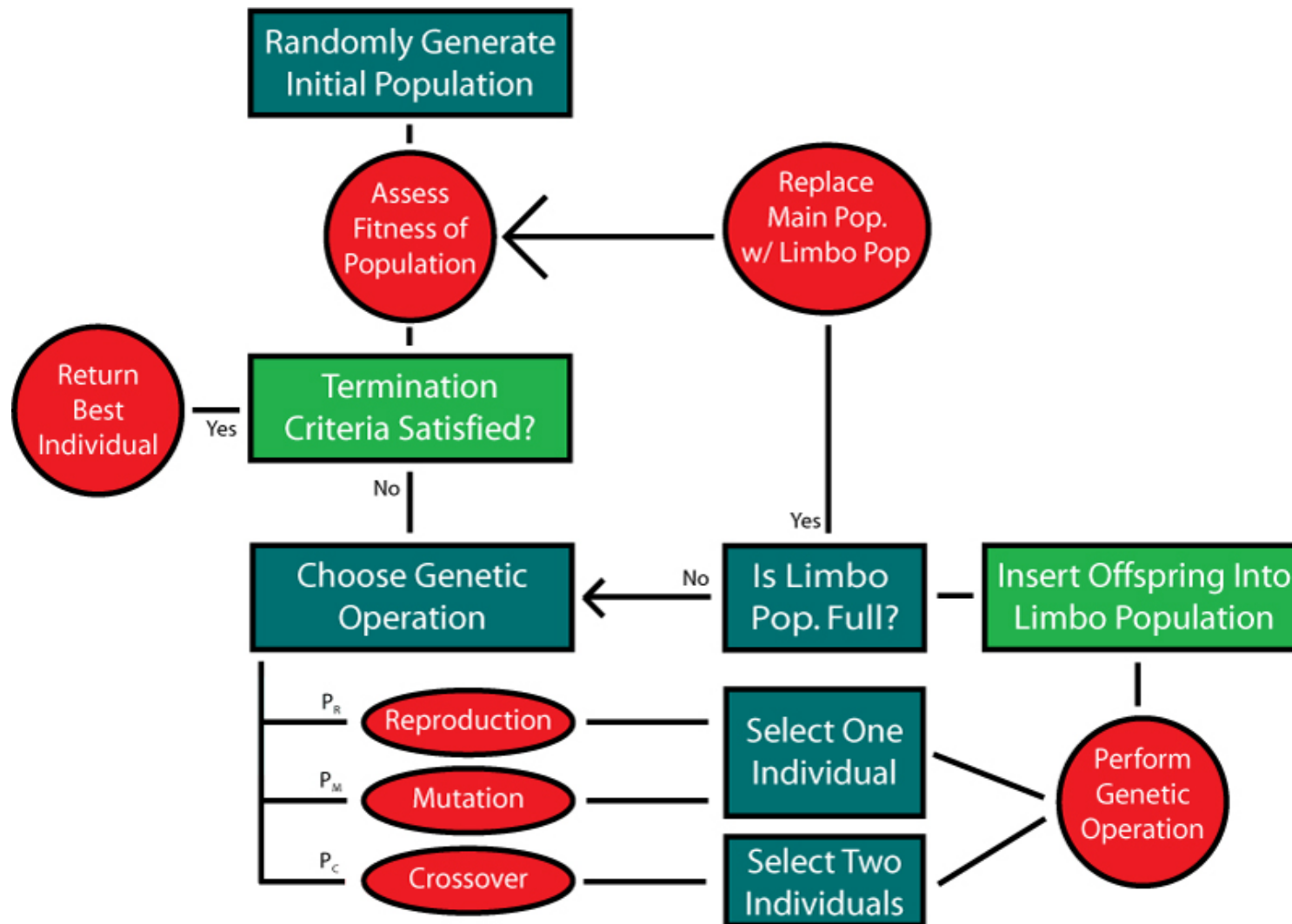
Presenter: Hung-Hsi Chen
Date: Mar. 02, 2010

Abstract

Classification is one of the important tasks in developing expert systems. Most of the previous approaches for classification problem are based on classification rules generated by decision trees. In this paper, we propose a new **learning approach based on genetic programming to generate discriminant functions for classifying data**. An adaptable incremental learning strategy and a distance-based fitness function are developed to improve the efficiency of genetic programming-based learning process. We first transform attributes of objects into fuzzy attributes and then a set of discriminant functions is generated based on the proposed learning procedure. The set of derived functions with fuzzy attributes gives high accuracy of classification and presents a linear form. Hence, the functions can be transformed into inference rules easily and we can use the rules to provide the building of rule base in an expert system.

Genetic programming(GP)

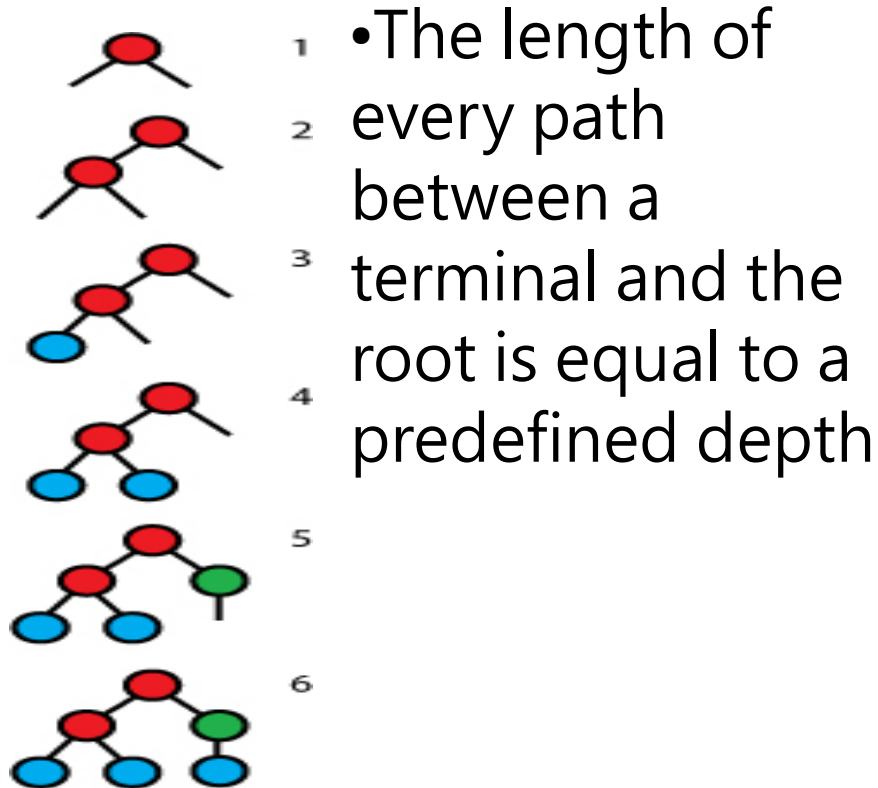
- Genetic programming is a branch of genetic algorithms.
- The [main difference](#) between genetic programming and genetic algorithms is [the representation of the solution](#).



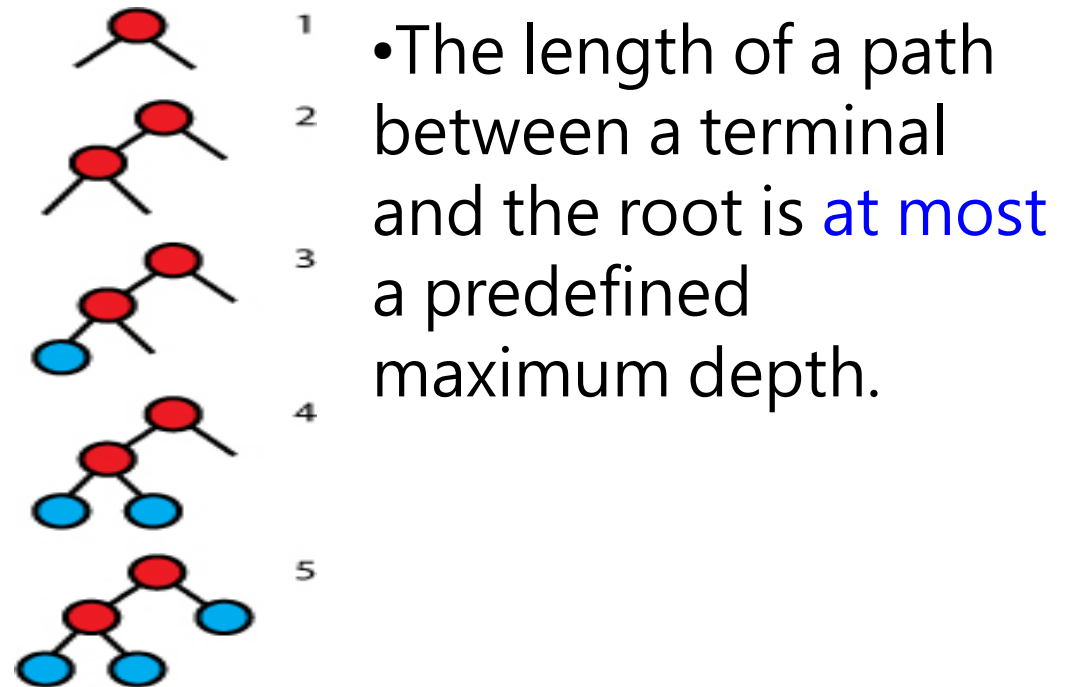
Initial population

- We have determined the **MAX DEPTH**

1 、 Full method

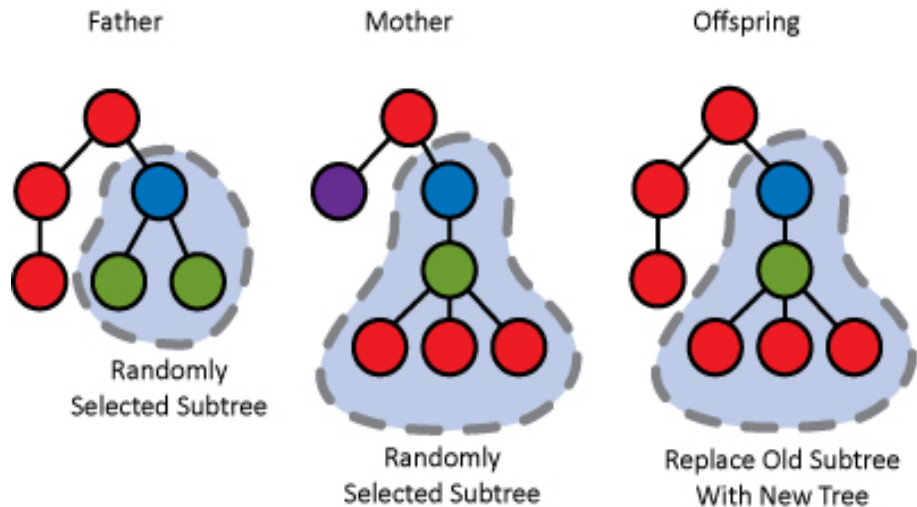


2 、 Grow method



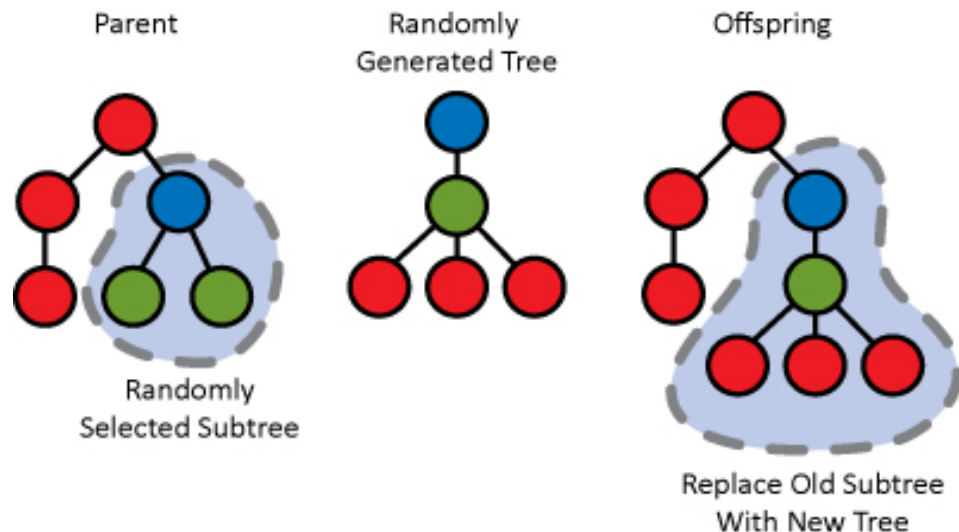
Usually, an equal mix of trees is produced with the Full and Grow methods.

Crossover(Subtree crossover)



The probability distribution for the selection of a crossover point **favors the exchange of subtrees rather than terminals.**

Mutation(Subtree mutation)



- First, a random point is chosen in the tree.
- This point, as well as the subtree below it, are then removed and replaced by a new, randomly generated subtree.**

GP as classifier

The classifier can be accomplished in either two ways:

1. classification rules
2. classification functions(discriminant functions)

The advantage of classifying by classification functions: instead of classification rules is **concise and efficient**.

However, the discriminant functions are hard to be understood and interpreted since the discriminant functions are **usually non-linear**.

Notations for classification problem with fuzzy attributes

dataset S , $x_j \in S$

x_j having n attributes $A_1, A_2, \dots, A_n \in \mathbb{R}$

$x_j = (v_{j1}, v_{j2}, \dots, v_{jn})$, where $v_{ji} \in A_i$

$$A_i = (\mu_{i1}/A_{i1} + \mu_{i2}/A_{i2} + \dots + \mu_{il_i}/A_{il_i})$$

where A_{ik} is one of the fuzzy sets

$u_{ik}: A_i \rightarrow [0, 1]$ is the membership function for A_{ik} , $1 \leq k \leq l_i$

$u_{ik}(v_{ji})$ is the fuzzy membership value of data x_j for fuzzy set A_{ik}

l_i is the number of fuzzy sets in attribute A_i

hence $x_j = (v_{j1}, v_{j2}, \dots, v_{jn}) \rightarrow$

$$y_j = (\mu_{11}(v_{j1}), \mu_{12}(v_{j1}), \dots, \mu_{1l_1}(v_{j1})),$$

$$\mu_{21}(v_{j2}), \mu_{22}(v_{j2}), \dots, \mu_{2l_2}(v_{j2}),$$

$$\dots, \mu_{n1}(v_{jn}), \mu_{n2}(v_{jn}), \dots, \mu_{nl_n}(v_{jn})$$

Classification problem with fuzzy attributes

K predefined classes denoted as $C_1, C_2, \dots, C_k, C = \{C_1, C_2, \dots, C_k\}$
 $\langle x_j, c_j \rangle \rightarrow \langle y_j, c_j \rangle$ is a fuzzy sample of class $c_j, c_j \in C$

training set $TS = \{ \langle y_j, c_j \rangle \mid y_j \text{ is fuzzy attributes, } c_j \in C, 1 \leq j \leq m \}$

$$m = |TS|, m = \sum_{i=1}^K m_i, 1 \leq i \leq K$$

discriminant function f_i for class $C_i, f_i: \mathbb{R}^{n'} \rightarrow \mathbb{R}, n' = \sum_{i=1}^n l_i$

$$f_i(y_j) \geq a, \text{ if } c_j = C_i$$

$$f_i(y_j) < a, \text{ if } c_j \neq C_i$$

The adaptable incremental learning strategy

Goal: obtaining solutions efficiently in GP

Parameter:

g : generations of evolution in each stage of the learning procedure

m' : the number of training samples in each stage

ρ : initial incremental rate of TS in each stage

α : adaptive factor

ω : condition of satisfying the effectiveness of fitness value

Where $0 \leq \rho \leq 1$, $0 \leq m' \leq m$, $\alpha \geq 1$

Main steps:

Step 1: $\alpha = 1$, $m'=0$. We give g , ρ , ω

Step 2: $m' = m \times \alpha \times \rho + m'$. If $m' \geq m$ then $m' = m$

Step 3: GP is evolved with the m' training sample for g generations.

A function will be obtained after the g generations.

Step 4: If $m' = m$ then stop else go to next step

Step 5: If the fitness value satisfies the condition ω
then $\alpha = 2$ else $\alpha = 1$. Go to step 2

Example

Stage1:

Step 1: $\alpha=1$, $m'=0$, $m=75$, $g=1000$, $\rho=0.2$, ω is the fitness value = 0

Step 2: $m' = m \times \alpha \times \rho + m' = 75 \times 1 \times 0.2 + 0 = 15$

Step 3: GP is evolved with $m'=15$ training samples for $g=1000$ generations

Step 4: $m'=15 \neq m=75$, go to next step

Step 5: if fitness value=0 then $\alpha=2$. Go to step 2. Starts next stage.

Stage2:

Step 2: $m' = m \times \alpha \times \rho + m' = 75 \times 2 \times 0.2 + 15 = 45$

Step 3: GP is evolved with $m'=45$ training samples for $g=1000$ generations

Step 4: $m'=45 \neq m=75$, go to next step

Step 5: if fitness value $\neq 0$ then $\alpha=1$. Go to step 2. Starts next stage.

Stage3:

Step 2: $m' = m \times \alpha \times \rho + m' = 75 \times 1 \times 0.2 + 45 = 60$

Step 3: GP is evolved with $m'=60$ training samples for $g=1000$ generations

Step 4: $m'=60 \neq m=75$, go to next step

Step 5: if fitness value $\neq 0$ then $\alpha=1$. Go to step 2. Starts next stage.

Stage4:

Step 2: $m' = m \times \alpha \times \rho + m' = 75 \times 1 \times 0.2 + 60 = 75$

Step 3: GP is evolved with $m'=75$ training samples for $g=1000$ generations

Step 4: $m'=75 \geq m=75$, the procedure stopped and obtained function.

Fitness function

The error degree for a positive instance is defined as

$$D_p = \begin{cases} 0, & \text{if } c_j = C_i \text{ and } f_i(y_j) \geq a, & \text{分類正確} \\ [p - f_i(y_j)]^2, & \text{if } c_j = C_i \text{ and } f_i(y_j) < a, & \text{分類錯誤} \end{cases}$$

and the error degree for a negative instance is defined as

$$D_n = \begin{cases} 0, & \text{if } c_j \neq C_i \text{ and } f_i(y_j) < a, & \text{分類正確} \\ [f_i(y_j) - q]^2, & \text{if } c_j \neq C_i \text{ and } f_i(y_j) \geq a, & \text{分類錯誤} \end{cases}$$

Thus, the fitness function is defined as

$$- \sum_{j=1}^{m'} (D_p + D_n),$$

We set $p = 100$, $q = -100$, $a = 0$ here.

Experiment

- Take the Fisher's Iris data set ([wiki](#))
- 150 training samples, 4 numerical attributes, 3 classification.
- The classification are [setosa](#)(山鳶尾), [versicolor](#)(變色鳶尾), [virginica](#)(維吉尼亞鳶尾)
- Original attributes are [sepal length](#)(萼片長度), [sepal width](#)(萼片寬度), [petal length](#)(花瓣長度), [petal width](#)(花瓣寬度) which are denoted [sl](#), [sw](#), [pl](#), [pw](#) respectively.
- We give tree terms '[short](#)', '[medium](#)', '[long](#)' as the fuzzy sets for each attribute.
- The fuzzy sets are [sl_S](#), [sl_M](#), [sl_L](#), [sw_S](#), [sw_M](#), [sw_L](#), [pl_S](#), [pl_M](#), [pl_L](#), [pw_S](#), [pw_M](#), [pw_L](#).



Table 1
Summary statistic of attributes in Iris set

	Maximum	Mean	Minimum
Sepal length	4.30	5.84	7.90
Sepal width	2.00	3.05	4.40
Petal length	1.00	3.76	6.90
Petal width	0.10	1.20	2.50

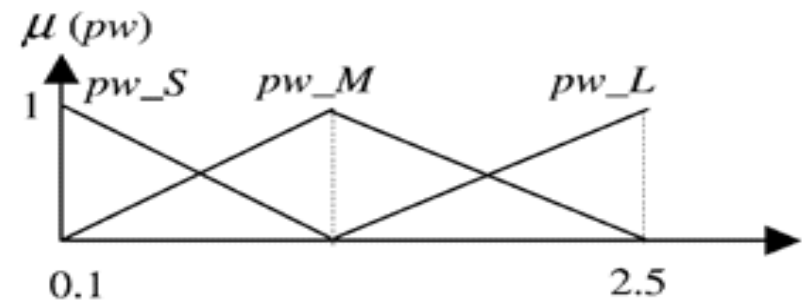
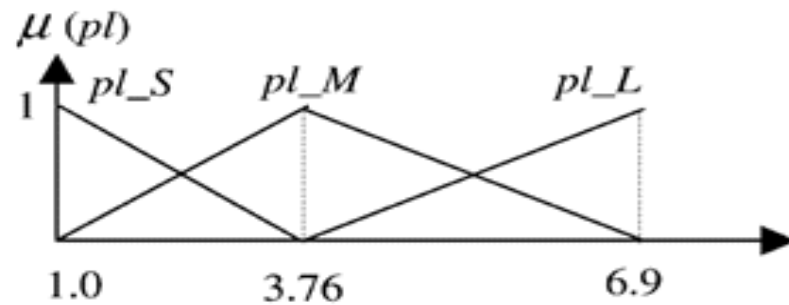
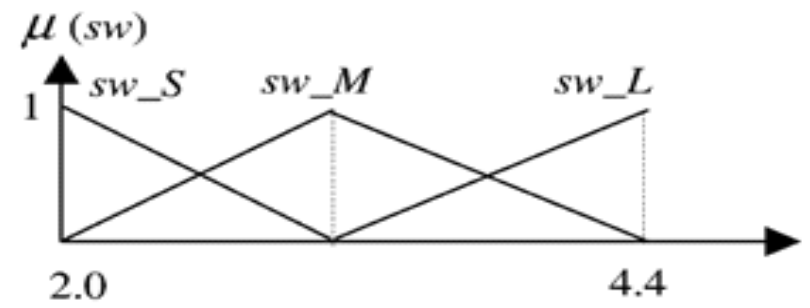
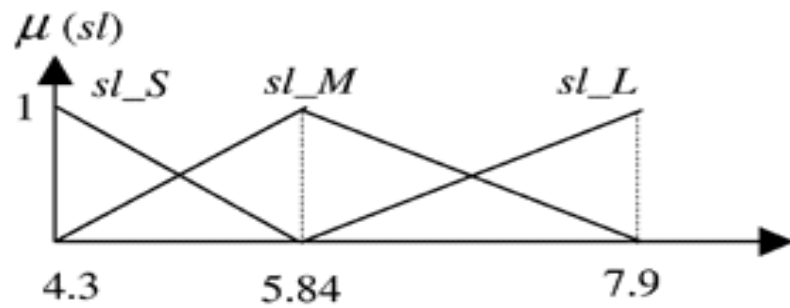


Fig. 3. The membership functions for Iris dataset.

GP parameters

Table 2

Parameters used in the experiment of Iris

Parameters	Values
Node, constant mutate weight	0.435
Mutation weight annealing	0.40
Incremental rate ρ	0.2
Population size	1000
Crossover weight	0.28
Crossover weight annealing	0.20
Adaptive factor α	2
Condition of fitness value	0
Generations per stage g	1000
Function set	$+$, $-$, $\times$, $\div$
Selection method	Tournament
Tournament size	7
Terminal set	$sl_L, sl_M, sl_S, sw_L, sw_M, sw_S,$ $pl_L, pl_M, pl_S, pw_L, pw_M, pw_S$

Experimental results

The linear discriminant functions that are obtained by fuzzy attributes:

$$f_{\text{setosa}} = (sl_S - pl_M + sw_L - pl_L)$$

$$f_{\text{versicolor}} = (29 \times pw_M - sl_M + 71 \times pl_M - 67)$$

$$f_{\text{virginica}} = (2 \times pw_L + pl_L - sw_L - sl_L - pl_M).$$

Judgment rules:

IF $(sl_S + sw_L \geq pl_M + pl_L)$

THEN setosa.

IF $(29 \times pw_M + 71 \times pl_M \geq 67 + sl_M)$

THEN versicolor.

IF $(2 \times pw_L + pl_L \geq sw_L + sl_L + pl_M)$

THEN virginica.

The discriminant functions that are obtained by original attributes:

$$f'_{\text{setosa}} = sw - pl$$

$$f'_{\text{versicolor}} = \frac{pl \times sl - \frac{pl}{-119} - \frac{-4 \times (pw - sl)}{pl}}{\frac{pw \times sl}{pl - sl} + 5 + sw + \frac{-16}{-104}}$$

$$f'_{\text{virginica}} = \frac{115}{29 \times pw + sl - 53}.$$

Non-linear and difficult for us to understand

Table 3

Experimental results with fuzzy attributes

	Fuzzy attributes		
	f_{setosa}	$f_{\text{versicolor}}$	$f_{\text{virginica}}$
Setosa	25	0	0
Versicolor	0	23	2
Virginica	0	0	25
Precision (%)	100	100	92.6
Recall (%)	100	92.0	100
Average accuracy (%)	97.3		
Overall accuracy (%)	97.3		

Table 4

Experiment results with original attributes

	Original attributes		
	f'_{setosa}	$f'_{\text{versicolor}}$	$f'_{\text{virginica}}$
Setosa	25	0	0
Versicolor	0	23	1
Virginica	0	0	23
Precision (%)	100	100	95.8
Recall (%)	100	92.0	92.0
Average accuracy (%)	94.7		
Overall accuracy	94.7		

N_i : the number of objects in class C_i

N_{f_i} : the number of objects recognized by function f_i (被分到第 i 類的個數)

$N_{f_i}^i$ the number of objects belonging to class C_i and recognized by function f_i
(被分到第 i 類且為正確分類的個數)

$$\text{Precision} = \frac{N_{f_i}^i}{N_{f_i}}$$

$$\text{Average accuracy} = \frac{1}{K} \sum_{i=1}^K \text{the recall of } f_i$$

$$\text{Recall} = \frac{N_{f_i}^i}{N_i}.$$

$$\text{Overall accuracy} = \frac{\sum_{i=1}^K N_{f_i}^i}{|S|}$$

Compare with other methods

Table 5

Accuracy comparison

Models or methods	Testing recognition rate (%)
GVS (Hong & Tseng, 1997)	96.0
FEBFC with 4 features (Lee et al., 2001)	96.7
FRG with GA (Lin & Chen, 2001)	96.87
FEBFC with 2 selected features (Lee et al., 2001)	97.1
FIL (Wang et al., 1999)	97.3
Our method	97.3

Thank you!!